Міністерство освіти і науки України
Львівський національний університет імені Івана Франка
Факультет електроніки та комп'ютерних технологій

Звіт про виконання лабораторної роботи №3
З дисципліни **«Функціональне програмування»**
На тему **«Нездатні до змін структури даних (immutability) у Python»**
Варіант 9

Виконав:
Студент групи ФЕП-23
Литвинюк Ярослав

Перевірив:
проф. Свелеба С. А.

Львів 2026

Мета роботи:

- зрозуміти та застосувати `tuple`, `frozenset`, `dataclasses(frozen=True)`, `MappingProxyType`;
- замінити мутації на повернення нових копій (pure updates);
- виміряти ефект (стабільність тестів, час виконання) після переходу на незмінні структури.

Завдання(варіант 9): Система доставки.

- **Незмінна модель:** Parcel, RoutePoint, labels, meta
- **pure-оновлення:** Додавання точки маршруту, мітки, зміна статусу
- **Що превірити:** Маршрут tuple; labels frozenset; meta read-only.

Хід роботи

requirements.txt

```
pytest>=8.0.0
mypy>=1.9.0
black>=24.2.0
ruff>=0.3.0
pytest-benchmark>=4.0.0
```

pyproject.toml

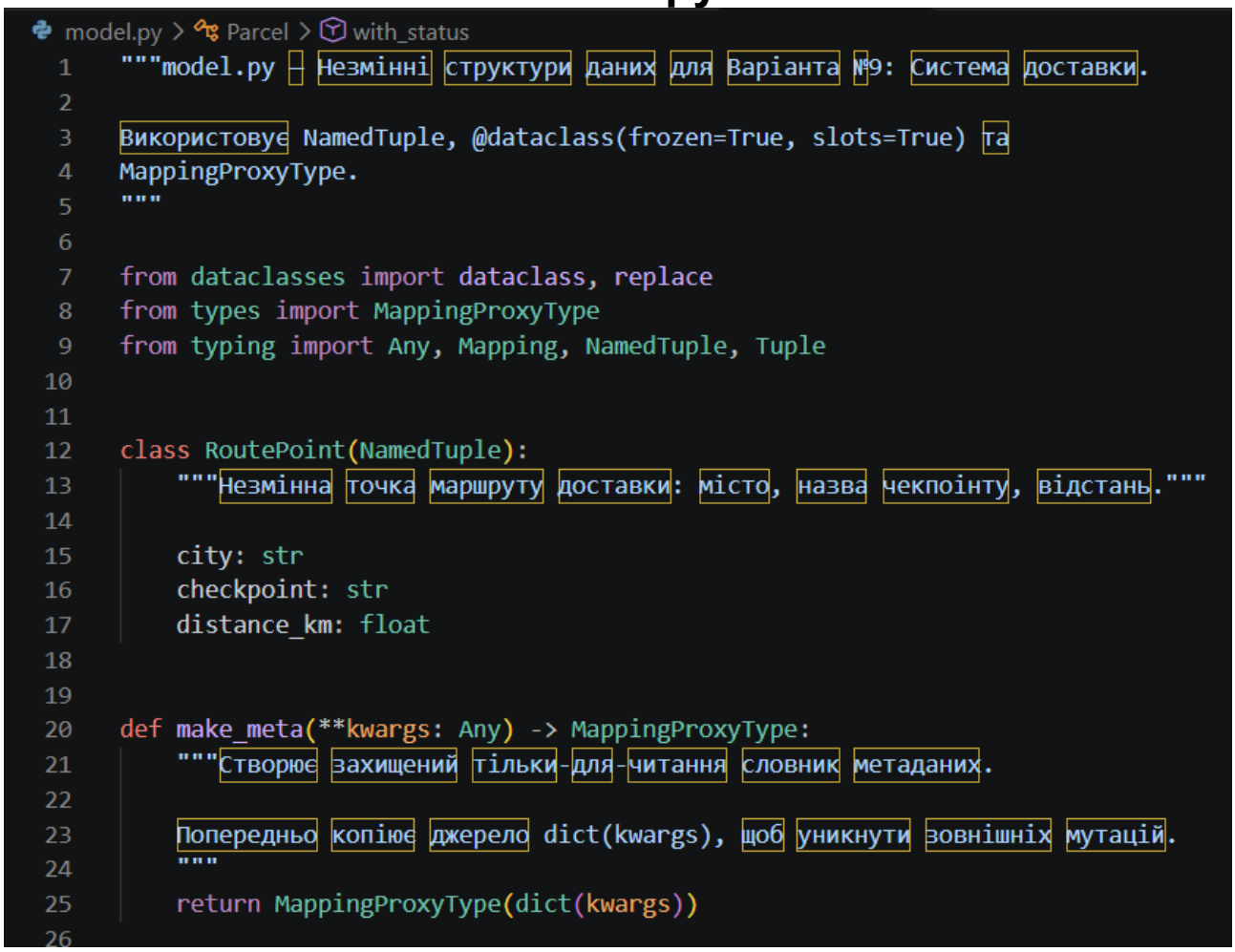
```
[tool.black]
line-length = 88
target-version = ["py310", "py311", "py312"]

[tool.ruff]
line-length = 88
target-version = "py310"
```

mypl.ini

```
[mypy]
python_version = 3.10
strict = True
disallow_untyped_defs = True
disallow_any_generics = True
ignore_missing_imports = True
```

model.py



```python
"""model.py — Незмінні структури даних для Варіанта №9: Система доставки.

Використовує NamedTuple, @dataclass(frozen=True, slots=True) та
MappingProxyType.
"""

from dataclasses import dataclass, replace
from types import MappingProxyType
from typing import Any, Mapping, NamedTuple, Tuple


class RoutePoint(NamedTuple):
    """Незмінна точка маршруту доставки: місто, назва чекпоінту, відстань."""

    city: str
    checkpoint: str
    distance_km: float


def make_meta(**kwargs: Any) -> MappingProxyType:
    """Створює захищений тільки-для-читання словник метаданих.

    Попередньо копіює джерело dict(kwargs), щоб уникнути зовнішніх мутацій.
    """
    return MappingProxyType(dict(kwargs))

```

```python
@dataclass(frozen=True, slots=True)
class Parcel:
    """Незмінна посилка в системі логістики.

    frozen=True генерує __hash__ і забороняє reassignment атрибутів;
    slots=True оптимізує пам'ять і пришвидшує доступ до полів.
    """

    id: int
    weight_kg: float
    status: str  # 'pending', 'in_transit', 'delivered', 'cancelled'
    route: Tuple[RoutePoint, ...]  # незмінний кортеж (НЕ list)
    labels: frozenset[str]  # незмінна множина міток (НЕ set)
    meta: Mapping[str, Any]  # read-only view MappingProxyType (НЕ dict)

    def total_distance(self) -> float:
        """Обчислює загальну дистанцію за всіма точками маршруту."""
        return round(sum(point.distance_km for point in self.route), 2)

    def with_route_point(self, point: RoutePoint) -> "Parcel":
        """Pure update: повертає новий Parcel із доданою точкою через конкатенацію tuple."""
        return replace(self, route=self.route + (point,))

    def with_label(self, label: str) -> "Parcel":
        """Pure update: повертає новий Parcel із новою міткою через об'єднання frozenset."""
        return replace(self, labels=self.labels | frozenset({label}))

    def with_status(self, new_status: str) -> "Parcel":
        """Pure update: повертає новий Parcel зі зміненим статусом через dataclasses.replace."""
        return replace(self, status=new_status)
```

core.py

core.py > freeze

```python
"""core.py — Pure-оновлення та бізнес-логіка розрахунків для системи доставки."""

from collections.abc import Mapping as MappingABC
from types import MappingProxyType
from typing import Any

from model import Parcel, RoutePoint


def add_route_point(
    parcel: Parcel, city: str, checkpoint: str, distance_km: float
) -> Parcel:
    """Чиста функція додавання нової точки маршруту."""
    point = RoutePoint(
        city=city, checkpoint=checkpoint, distance_km=distance_km
    )
    return parcel.with_route_point(point)


def add_label(parcel: Parcel, label: str) -> Parcel:
    """Чиста функція додавання логістичної мітки (наприклад, 'fragile', 'express')."""
    return parcel.with_label(label)


def change_status(parcel: Parcel, new_status: str) -> Parcel:
    """Чиста функція зміни статусу посилки."""
    return parcel.with_status(new_status)

```

```python
def calculate_delivery_cost(
    parcel: Parcel,
    *,
    base_rate_per_km: float = 0.8,
    weight_factor: float = 1.2,
) -> float:
    """Чиста функція розрахунку вартості доставки з урахуванням міток і дистанції."""
    dist = parcel.total_distance()
    cost = dist * base_rate_per_km + (parcel.weight_kg * weight_factor * 10)

    if "express" in parcel.labels:
        cost *= 1.3
    if "fragile" in parcel.labels:
        cost += 50.0

    return round(cost, 2)


def freeze(obj: Any) -> Any:
    """Рекурсивне глибоке заморожування вкладених структур (Deep Freeze шаблон)."""
    if isinstance(obj, MappingABC):
        frozen_inner = {k: freeze(v) for k, v in obj.items()}
        return MappingProxyType(frozen_inner)
    if isinstance(obj, set):
        return frozenset(freeze(x) for x in obj)
    if isinstance(obj, (list, tuple)):
        return tuple(freeze(x) for x in obj)
    return obj
```

app.py

app.py > ...

```python
"""app.py — Демонстраційний модуль (Imperative Shell).

Ілюструє життєвий цикл незмінної посилки, pure updates та захист від мутацій.
"""

from dataclasses import FrozenInstanceError

from core import (
    add_label,
    add_route_point,
    calculate_delivery_cost,
    change_status,
    freeze,
)
from model import Parcel, RoutePoint, make_meta


def render_parcel_info(title: str, p: Parcel) -> None:
    """I/O функція форматованого відображення стану посилки."""
    print("-" * 68)
    print(f"[{title}]")
    print(f"• ID: {p.id} | Статус: {p.status.upper()} | Вага: {p.weight_kg} кг")
    print(
        f"• Мітки (Labels): {', '.join(sorted(p.labels)) if p.labels else 'немає'}"
    )
    print("• Маршрут доставки:")
    for idx, pt in enumerate(p.route, start=1):
        print(f"   {idx}. {pt.city} ({pt.checkpoint}) — {pt.distance_km} км")
    print(f"• Загальна дистанція: {p.total_distance()} км")
    print(f"• Орієнтовна вартість: ${calculate_delivery_cost(p):.2f}")
    print(f"• Метадані (meta): {dict(p.meta)}")

```

```python
def main() -> None:
    print("=" * 68)
    print(
        "       СИСТЕМА ДОСТАВКИ ПОСИЛОК: ІМУТАБЕЛЬНА МОДЕЛЬ (ВАРІАНТ №9)      "
    )
    print("=" * 68)

    # 1. Створення початкової незмінної посилки
    start_point = RoutePoint(
        city="Львів", checkpoint="Відділення №1", distance_km=10.0
    )
    initial_parcel = Parcel(
        id=9001,
        weight_kg=4.2,
        status="pending",
        route=(start_point,),
        labels=frozenset({"standard"}),
        meta=make_meta(
            sender="Ярослав",
            recipient="Галина",
            declared_value_uah=2500,
            service="NovaPost",
        ),
    )
    render_parcel_info("1. СТВОРЕННЯ ПОСИЛКИ", initial_parcel)

    # 2. Pure update: додавання точок маршруту
    p_route = add_route_point(
        initial_parcel, "Рівне", "Транзитний HUB-Захід", 210.0
    )
    p_route = add_route_point(
        p_route, "Київ", "Центральне відділення №14", 330.0
    )
    render_parcel_info("2. ПІСЛЯ ДОДАВАННЯ ТОЧОК МАРШРУТУ", p_route)

    # 3. Pure update: додавання міток
    p_labeled = add_label(p_route, "fragile")
    p_labeled = add_label(p_labeled, "express")
    render_parcel_info("3. ПІСЛЯ ДОДАВАННЯ МІТОК (FRAGILE, EXPRESS)", p_labeled)

    # 4. Pure update: зміна статусів
    p_transit = change_status(p_labeled, "in_transit")
    p_delivered = change_status(p_transit, "delivered")
    render_parcel_info("4. ФІНАЛЬНИЙ СТАН (DELIVERED)", p_delivered)

    # 5. Демонстрація збереження початкового об'єкта
    print("-" * 68)
    print("[ПЕРЕВІРКА НЕЗМІННОСТІ ПОЧАТКОВОГО ОБ'ЄКТА]")
    print(f"• Початковий ID 9001 статус: {initial_parcel.status}")
    print(
        f"• Точок у початковому маршруті: {len(initial_parcel.route)} (у фінальному: {len(p_delivered.route)})"
    )
    print(f"• Початковий об'єкт is фінальний: {initial_parcel is p_delivered}")
```



```
    # 6. Перевірка рантайм-захисту від мутацій
    print("-" * 68)
    print("[ТЕСТУВАННЯ ЗАХИСТУ ВІД МУТАЦІЙ У РАНТАЙМІ]")
    try:
        initial_parcel.status = "hacked"  # type: ignore
    except FrozenInstanceError as e:
        print(
            f"✅ Захист @dataclass(frozen=True): {type(e).__name__} — пряма мутація заборонена!"
        )

    try:
        initial_parcel.meta["recipient"] = "Hacker"  # type: ignore
    except TypeError as e:
        print(f"✅ Захист MappingProxyType: {type(e).__name__} — {e}")

    # 7. Глибоке заморожування конфігурації
    raw_config = {
        "tariff_zones": ["West", "Center", "East"],
        "discounts": {"business": 0.15},
        "flags": {"auto_notify", "insurance"},
    }
    frozen_cfg = freeze(raw_config)
    print("-" * 68)
    print(
        f"✅ freeze() успішно створив: {type(frozen_cfg).__name__} "
        f"з вкладеними {type(frozen_cfg['tariff_zones']).__name__} та {type(frozen_cfg['flags']).__name__}"
    )
    print("=" * 68)


if __name__ == "__main__":
    main()
```

Висновок: Досліджено проектування незмінних структур даних засобами NamedTuple, @dataclass(frozen=True, slots=True), frozenset та MappingProxyType. Підтверджено переваги підходу *pure updates* над *in-place* мутаціями: усунення прихованих побічних ефектів і потокобезпечність ціною мінімальних накладних витрат на копіювання.